

Verification Is Not Acceptance: Evidentiary Sufficiency and Settlement Authority in Contracted Compute Delivery

Fée-Fée Yang

feifei.gelato@gmail.com

August 2026

Contents

Abstract	1
1. Introduction	2
2. How Compute Delivery Is Reconciled Today	4
3. Threat Model	5
4. Related Work	7
5. The Separation Principle	9
6. Formal Model	14
7. System Design	18
8. Implementation	23
9. Evaluation	23
10. Limitations	29
11. Open Problems	31
12. Conclusion	32
References	33

Abstract

Contracted compute is bought in units of hundreds of millions of dollars, and the industry has no shared mechanism for establishing whether a delivery obligation was met. Disputes over availability, capacity, and degradation are resolved by reconciling provider telemetry against customer logs by hand, over weeks, leaving no reproducible record of what was decided or on what basis. Recent proposals close this gap by compiling contractual intent into machine-checkable obligations and settling on an automated verification verdict.

We argue that the final step is a category error. Verification establishes evidentiary sufficiency, a question about whether a body of evidence supports a stated claim under a stated standard. Acceptance is an exercise of delegated commercial authority, a question about whether a party agrees to be bound and to pay. The two have different correctness conditions, and a verdict produced by the first cannot discharge the second. Systems that conflate them generate a result no counterparty is obliged to honor.

We present PSYCHE, a delivery-accounting system built on the separation. PSYCHE com-

piles a contract into a privacy-preserving Contract Profile, admits evidence through a gate that treats provenance, coverage gaps, and inter-party conflict as first-class outcomes, assembles the admitted set into a signed and replayable Proof-of-Reality Capsule, and evaluates that Capsule against the Profile to produce a support assertion. Then it halts. A settled state is reachable only through a counterparty-signed review.

We give a formal model of the system as a fold over an append-only signed log. Its central property is acceptance soundness: a settled state implies a counterparty-signed review issued inside a valid grant. Four supporting properties make that guarantee usable — confluence under causally consistent reordering, terminal monotonicity, equivocation detectability, and temporal binding. The last two hold against an untrusted fabric operator, which we take to be a requirement rather than a refinement, since a settlement system whose operator can rewrite or fork the record has no neutrality to offer. We further decompose finality, normally treated as one event, into four finalities established independently — evidence, measurement, counterparty, commercial — and show that verification-driven settlement acquires its authority by substituting the first two for the third. Prior verification-native clearing designs appear in our model as the degenerate instantiation in which the review authority is the verifier.

We describe an implementation of the full contract-to-archive pipeline and evaluate reproducibility from the replay bundle alone, invariant enforcement under adversarial and concurrent orderings, the behavior of a four-valued verification output against a two-valued baseline, and a worked partial-delivery dispute. Results are reported over a synthetic corpus.

1. Introduction

A GPU cloud invoices a customer for 1,000 H100-hours delivered over thirty days. The customer's scheduler shows 972 hours of usable capacity, 28 hours during which memory bandwidth ran below the contracted floor, and three nodes that never matched the contracted SKU at all. Both parties have logs. Each set of logs is internally consistent. Neither party can compel the other to accept its numbers. The contract provides for service credits and specifies that credits are the sole and exclusive remedy. Six weeks later a number is agreed on a call, and nobody writes down how it was reached.

This is the ordinary case. It repeats across an asset class that has grown faster than the institutions that would normally govern it. Several neocloud providers offer no standing uptime SLA in default terms, and some decline to publish one at all, negotiating it per deal [1]. Where an SLA exists, the measurement period, the exclusion list, the liability cap, and the definition of availability itself differ between the datacenter-operator contract and the operator-customer contract, so an operator may recover a fraction of monthly rent from the facility beneath it while owing credits computed against far larger service commitments above [2]. As of mid-2026 no court has issued a final judgment on the merits in a pure AI-cloud SLA dispute [1]. The market has no case law, no settled practice, and no neutral record. It has bilateral reconciliation conducted in spreadsheets.

The natural engineering response is to make the obligation machine-checkable: compile the contract into a structured object, bind execution artifacts to it, verify automatically, settle on the result. Recent work develops this program carefully [3]. We accept nearly all of it and

part company at the final step.

1.1 Thesis. Verification and acceptance are different operations, and a system that settles must keep them apart.

Verification asks whether a body of evidence supports a claim, under a given standard, over a given window. The question is one of fact. Its correctness condition is soundness with respect to the evidence admitted and the standard applied. It can be automated, audited, and recomputed by anyone holding the same inputs.

Acceptance asks whether the obligated party agrees to be bound and to pay. The question is one of authority. Its correctness condition is not truth but authorization: whether the decision was taken by someone empowered to take it, inside their limits, on the record. No quantity of evidence produces it, because acceptance is a fact about a party's will rather than about the world.

A verification verdict, however sound, creates no obligation to pay. When a counterparty declines to honor it, the system has recorded a disagreement and called it a settlement. The conflation also carries an adoption cost we take to be decisive: a finance function will not delegate acceptance authority over material spend to a third-party algorithm, and counsel will not advise it to. The party who must be persuaded to join such a network is the party asked to surrender the most.

We did not arrive at the separation reluctantly. It is the property that makes the system deployable, and it describes how durable settlement institutions have always been built. An inspection certificate, a bill of lading, a warehouse receipt, and a clearing member's position report are evidentiary instruments produced under agreed procedure. None of them is the moment of acceptance. Their value lies in constraining what that moment can reasonably contain.

1.2 A second commitment: the fabric is not trusted. A settlement system holds the canonical record. Whoever operates it can, absent countermeasures, rewrite that record, drop its tail, or present one history to the buyer and a different history to the seller. Hash chaining alone does not prevent the last of these, since a party controlling the chain head can maintain two internally consistent chains.

We therefore treat the fabric as untrusted for integrity and rely on it only for availability. Integrity comes from participant signatures, a transparency-log construction over the event log, and inclusion and consistency proofs that participants exchange. The commitment pays a second dividend. It also settles the timing problem. The argument of §5.1 rests entirely on evidence being committed before its author knows which way the dispute will run, and a published log head supplies exactly that bound without a trusted timestamp authority (§6.6).

1.3 Contributions.

1. We state the separation principle and argue that conflating evidentiary sufficiency with settlement authority has concrete legal and commercial consequences (§5). We show that the distinction concerns authority rather than staffing, and therefore survives — and becomes load-bearing under — full automation of both counterparties (§5.6).
2. We give a **threat model** in which the fabric operator, both counterparties, and the evidence sources are all untrusted, and we state the security goals and non-goals that

follow (§3).

3. We give a **formal model** of the system as a fold over an append-only signed log. Its central property is acceptance soundness; confluence, terminal monotonicity, equivocation detectability, and temporal binding support it, the last two against an untrusted log operator (§6). A table maps each formal object to the module implementing it and the tests holding it (§6.9). Verification-driven clearing appears as the degenerate instantiation in which the review authority is the verifier (§6.10).
4. We formalize the separation as **non-derivation invariants**, stated negatively as an enumeration of the objects that may not produce a settled state, and enforced in the event model rather than in application policy (§5.3).
5. We decompose finality into **four finalities established independently** and show that verification-driven settlement acquires its authority by substituting the first two for the third (§5.4).
6. We describe the design and implementation of **PSYCHE** (§7–§8), and report three results from evaluating it (§9). Identical admitted evidence settles differently under two pinned standards, which locates the meaning of a delivery record in the standard rather than in the evidence. On a worked contract, the amount a two-valued verifier would allocate by default on a Case nobody decided is half the contracted period. And the enumerated invariants caught a defect in our own acceptance check, which is the argument for enumerating them.

Every table in §9 is generated by a harness in the repository and can be regenerated from a clean checkout.

§2 develops the problem domain. §3 gives the threat model and §4 surveys related work. §5 develops the principle, §6 the formal model, §7 the system design, §8 the implementation. §9 evaluates, §10 states limitations, §11 discusses open problems, §12 concludes.

2. How Compute Delivery Is Reconciled Today

2.1 An obligation that resists observation. Contracted compute has an awkward property. It is a service whose delivery is continuous and high-dimensional, and it is observed principally by parties with an interest in the observation.

A commitment for N accelerators of type T , available at rate R , over window W , is not met or missed at an instant. It is met to a degree along several axes at once: instance availability, interconnect health, thermal behavior, memory bandwidth, storage throughput, scheduler queue latency. Each axis can sit inside tolerance while the delivered value falls outside it. Contracts respond by defining an availability metric, a measurement period, an exclusion list, and a credit schedule. The metric projects a high-dimensional signal onto a scalar, and the projection is where the disputes live.

Both sides instrument. The provider holds node telemetry, hypervisor logs, and fabric counters. The customer holds scheduler records, job exit codes, and application throughput. These are not two views of one event stream. They are collected against different clocks, aggregated over different windows, and retained under different policies. Establishing that a provider-reported degradation and a customer-reported job failure describe the same event

is itself a reconciliation problem, and it is usually the first one the parties hit.

2.2 The remedy structure suppresses the record. The commercial structure compounds the difficulty. Service credits are typically the sole and exclusive remedy, capped at a percentage of fees for the affected period, and available only on a documented request filed inside a claim window [1]. Three consequences follow.

The burden of proof falls on the party with the weaker instrumentation. A customer must document a shortfall in the provider's service using data the customer gathered from outside the provider's infrastructure.

The cap makes rigor irrational. When the maximum recoverable credit is a small fraction of monthly spend and constructing the claim costs engineering weeks, most shortfalls go unclaimed. The absence of disputes in this market is therefore not evidence about delivery quality. It is evidence that disputing does not pay.

Because credits are exclusive, disagreements never reach a forum that would generate precedent. They are settled privately, and the settlement terms are confidential. This is the mechanism behind the observation that no AI-cloud SLA dispute has produced a merits judgment [1]. The market has no case law because its remedy structure is built to prevent cases from forming. Each dispute is resolved as though it were the first one, and the knowledge generated in resolving it leaves with the people who were on the call.

2.3 The mismatch propagates. The supply chain is layered. A facility operator sells to a neocloud, which may sell through a broker or marketplace, which sells to a buyer. Each layer negotiates its SLA independently, and the definitions do not align across layers [2]. Measurement periods differ. Exclusion lists differ. Liability caps differ. Availability itself is defined differently at each interface, so an outage that counts at one layer may not count at the next.

The obligations therefore do not net. An operator can owe customer-facing credits computed on service contract value while recovering a percentage of monthly rent from below, and no participant holds a record in which the two can be placed side by side.

High value, high volume, many layers, independent bilateral reconciliation, incompatible definitions, no shared record. This is the condition under which clearing infrastructure has historically emerged in other markets, and it has not emerged here.

2.4 The gap is narrower than trust. Exactness pays here, because the honest answer is smaller and more tractable than the word *trust* suggests.

The parties are not, in the main, lying. In the disputes whose structure we have been able to examine, both sides' data are accurate as far as they go. Four things are absent.

A pinned standard. The contract's availability definition lives in prose, is read differently by each side, and is not bound to the measurement at the time the measurement is taken.

Contemporaneous commitment. Evidence is assembled after a dispute arises, out of logs retained for operational reasons, at aggregation levels chosen for operations rather than for proof.

An account of what was not observed. Gaps carry most disputes, and current practice offers no object for the statement *we hold no data for this interval, and here is why*.

A reproducible derivation. The final number comes out of a negotiation. It cannot be re-

computed, cannot be audited, and cannot be compared against the number produced by any other dispute.

None of the four requires a disinterested party to watch the delivery. They require each party's own observations to be committed against a shared standard before anyone knows there will be a dispute. That requirement drives everything in §7.

3. Threat Model

The system's purpose is to produce a record two adversarial parties will both rely on. We therefore assume no participant is honest, including the participant that runs the system.

3.1 Participants and trust assumptions.

Participant	Trusted for	Not trusted for
Operator (claimant, Capsule producer)	nothing	evidence completeness, claim accuracy, timing of its own records
Customer (counterparty, review signer)	nothing	evidence completeness, good-faith review
Evidence source	nothing beyond its declared class	independence claims, coverage claims
Standards Registry	integrity of published standards and Profile schemas	commercial terms, which it never receives
Fabric (the PSYCHE deployment)	availability, ordering service	integrity, single-history presentation, non-censorship
Independent verifier	nothing	correctness, which is recomputable by anyone

The fabric's position is the load-bearing assumption. We rely on it to accept events, assign sequence, and serve reads. We do not rely on it to be honest about what it accepted. A deployment run by one of the counterparties is admissible under this model, which matters because §9.5 argues that the minimal useful configuration is a single-tenant one.

3.2 Adversary capabilities.

	Capability
A1	Withholding. A party declines to submit evidence in its possession.
A2	Selective disclosure. A party submits only the intervals favorable to it.
A3	Backdating. A party signs a record asserting a time earlier than the actual time of creation.
A4	Post-hoc revision. A party alters a record after signing it.
A5	Equivocation. The fabric presents log L to one participant and a divergent log L' to another.
A6	Truncation. The fabric drops a suffix of the log, or censors an event, and serves the result as complete.
A7	Authority overreach. A participant signs a review outside the scope of its grant, or after revocation.
A8	Replay. A participant resubmits a previously signed event to obtain a second effect.
A9	Collusion. Two participants coordinate against a third, including the fabric colluding with a counterparty.

	Capability
A10	Coordinated fabrication. Both counterparties contemporaneously sign, in the pre-agreed form, records of an event that did not occur.

We assume signature forgery and hash collision are infeasible, and we do not model key compromise. A10 is out of scope and we say why in §3.4.

3.3 Security goals. G1 Acceptance authenticity. A derived accepted implies a counterparty-signed accept, issued inside a valid grant, with quorum satisfied. Formalized as Theorem 1.

G2 Non-equivocation. A fabric that presents divergent histories to two participants can be proven to have done so, from evidence those participants hold. Theorem 4.

G3 Evidence integrity. Alteration of admitted evidence after packaging is detectable by any holder of the Capsule. Addressed by the digest chain (§7.4).

G4 Temporal binding. Evidence admitted to a Case is provably committed before a published bound, without reliance on the signer’s asserted clock. Theorem 5. This defeats A3.

G5 Coverage honesty. An unobserved interval cannot be silently counted as observed in either direction. Addressed by the BoundaryRecord and the coverage check (§7.3).

G6 Authority containment. A decision exceeding its grant is detectable from the log alone, without consulting the granting party. Addressed by grant binding in the review event (§7.6).

G7 Recomputability. Any party holding the ReplayBundle derives the same result as the originating evaluation. Definition 6 and §9.1.

3.4 Non-goals. Ground truth. The system does not establish that delivery occurred. It establishes what evidence was committed, under which standard it was evaluated, and who accepted the result. §5.3 argues that a system claiming more would be claiming what it cannot discharge.

Coordinated fabrication (A10). Two parties who both sign a false record contemporaneously defeat the design. Contemporaneous commitment raises the cost, since each must commit before learning which fabrication serves it, and non-repudiation raises the consequence once discovered. Neither eliminates the attack. We doubt any system at this layer can, and a design that claimed otherwise would be relying on an unstated assumption about at least one party’s honesty.

Availability under fabric denial of service. A fabric that refuses all writes halts the system. Liveness of that kind is an operational concern rather than a protocol one. Censorship of a *particular* event is a different matter and is covered by A6 and Theorem 4.

Confidentiality of evaluable structure from admitted verifiers. Verifiers necessarily learn the thresholds they evaluate against. §7.2 gives the disclosure boundary and states what remains open.

Compelling acceptance. A counterparty may refuse. This is by construction, and §5.5 explains why a system that could compel acceptance would not be one anybody joined.

4. Related Work

Verification-native clearing. The closest work is RAILS [3], which develops a clearinghouse protocol for agentic commerce over seven primitives, among them an Obligation Object compiling natural-language intent into a signed machine-clearable contract, an Evidence Envelope binding execution artifacts to that obligation under explicit admissibility classes, and a Verification Mesh applying several verifier types under conflict-resolution and escalation rules, yielding a Clearing Decision and a Settlement Instruction.

The two papers address different problems with overlapping machinery, and the difference is worth naming. RAILS is concerned with the principal and its agent: an agent may hold valid authorization, settle a valid payment, and receive exactly what it asked for, and still leave the user harmed, because it asked for the wrong thing. We are concerned with the two sides of a contract: whether the counterparty delivered what was agreed, and who is entitled to say so. Compiled obligations, bound evidence, and layered verification serve both. We read the independent convergence of that work with our own, on nearly identical primitives, as evidence that the underlying problem is structural: two efforts from different premises arrived at compiled obligations, bound evidence envelopes, and layered verification because the problem admits few other decompositions. RAILS also states a property of the kind we think this area needs: no financially material settlement rests on evidence below the obligation’s admissibility floor, offered as falsifiable against the specification rather than proved [3].

We adopt that framing and diverge at one point, which the paper makes explicit enough to quote. Where the mesh cannot reach its confidence threshold, RAILS escalates to a human arbiter, and *the arbiter is itself a verifier in role-tag terms, its output re-entering adjudication as one more verifier vote* under the same aggregation rules as any machine verifier [3, §6.3]. A party dissatisfied with a provisional Settlement Instruction may appeal, and the appeal *returns the obligation to verification with whatever new evidence it introduces*, after which a new Clearing Decision must be emitted before finality. The obligated party therefore never decides; it submits evidence to the party that decides. This is a coherent design and we take it to be the wrong allocation, for the reasons of §5. §6.10 states the relationship in the model’s terms: the two designs differ in the value of one parameter rather than in kind.

Verify-then-pay settlement. TessPay [19] carries the same allocation to its limit. Funds are escrowed at delegation, a Proof of Task Execution accumulates evidence during execution, and the escrow releases when that proof satisfies the obligation’s verification predicates. No party accepts; a predicate does. The design is admirably clean, and it makes the question this paper asks unavoidable, because a predicate that moves money is exercising an authority nobody granted it.

Tamper-evident logging and transparency. Our non-equivocation construction follows Certificate Transparency [7], which appends to a Merkle tree [16], publishes signed tree heads, and supplies inclusion and consistency proofs so that a log presenting divergent views can be caught by clients who compare heads. Crosby and Wallach’s history trees [8] give the data structure for tamper-evident logs with efficient membership and incremental proofs, and their threat model — an untrusted logger, auditors who verify — is the one we adopt. Haeberlen et al.’s PeerReview [9] establishes accountability in distributed systems by making every node’s actions verifiable against a signed log, and its notion of exposing a faulty node through conflicting signed statements is the mechanism behind Theorem 4. Haber and Stornetta’s timestamping construction [12] is the ancestor of our temporal bind-

ing result, and RFC 3161 [13] is the alternative we avoid, since a timestamp authority reintroduces a trusted party we have otherwise removed.

Fair exchange and non-repudiation. The problem of two mutually distrustful parties exchanging items with a guarantee for both has a long literature. Pagnia and Gärtner [11] establish that fair exchange is impossible without a trusted third party, and optimistic protocols [10] respond by involving the third party only when a party misbehaves. Our setting is a variant: the item exchanged on one side is a delivery and on the other a payment, the delivery is not atomically transferable, and the third party is present but untrusted for integrity. We do not solve fair exchange. We narrow the residue to a single signed decision by an identified party, which is what the impossibility result implies is the most one can do without trusting somebody.

Runtime verification and attested execution. A substantial literature addresses whether a computation ran as specified, through trusted execution environments, remote attestation, and runtime verification of governed agent actions [4], as well as server-side verification of tool-call claims independent of model-generated rationales [5]. This work sits beneath us and complements us. PSYCHE treats attestation output as evidence entering an admissibility gate and remains deliberately agnostic about which proof system produced it. Attestation can establish that a computation occurred with certain properties. Whether those properties satisfy a commercial obligation is a question about a contract, and no attestation answers it.

Authenticated data feeds. Where evidence originates outside the parties, the question becomes how an external observation is bound to a claim without trusting whoever relayed it. Town Crier [17] answers it with a trusted execution environment that fetches data and signs an attestation an on-chain contract can check, and DECO [18] answers it without special hardware, letting a prover convince a verifier that a TLS session returned a particular value. Our oracle challenge transcript occupies the same position and carries a narrower obligation: it establishes that a challenge was issued, answered, and bound to an issuance log head, and it makes no claim about the truth of the answer. That is a deliberate weakening, for the reason §5.3 gives about Sigma.

Replicated state machines. The fold of §6 is a deterministic state machine over an ordered log, and Theorem 2 is a confluence property of the kind that replicated state machine designs establish for their transition functions [14]. We do not replicate for fault tolerance. Participants derive state independently from a log they can each verify, which yields agreement without consensus among them, at the cost of relying on the fabric for ordering and detecting rather than preventing its misbehavior.

Provenance. Our evidence model records producer, method, and time in a manner close to the PROV data model [15], and a Capsule can be read as a provenance graph over a claim. We depart in requiring provenance assertions to be signed by the asserting party and admitted through a gate that can reject them.

Cloud SLA monitoring and compliance. A long line of work addresses SLA specification languages, monitoring architectures, and automated violation detection for conventional cloud services. Three features distinguish the present setting. The contracted unit is scarce and expensive enough to make individual disputes material. The observable is multi-dimensional, and degradation is continuous rather than binary. The customer's own instrumentation is a primary evidence source rather than a monitoring afterthought. We

depart from that literature further in treating the output of monitoring as inadmissible for settlement in the absence of an authorized acceptance act.

5. The Separation Principle

5.1 Why self-interested records can bind. Any evidence-based settlement design meets the same objection early: the primary evidence is produced by an interested party, so it cannot be trusted. The objection proves too much, because it applies with equal force to instruments that have worked for centuries.

A bill of lading is issued by the carrier. The carrier is party to the contract of carriage and has an evident stake in what the document says. Banks release payment against it anyway, under letters of credit, in transactions where no disinterested party stood at the quayside and counted.

The instrument works because of when it is signed. At the moment of loading, the carrier does not know whether the cargo will arrive damaged, whether the buyer will find a reason to refuse it, or which way its own interest will eventually run. A record made under that ignorance carries information. A record composed afterward, by a party who has since learned how the dispute is going, carries much less, and every participant knows it.

Three properties do the work.

The record is **contemporaneous**, made at the time of the event and before any dispute exists.

Its **form is fixed in advance**, by the parties or by convention, so the issuer cannot choose after the fact what to record and what to omit.

It is **signed, and alteration is detectable**, so the issuer cannot revise the record once its interest becomes clear.

What these buy is a weaker guarantee than trust, and the weakness is the point. No participant is asked to believe the carrier honest. The system asks only that the carrier commit early, in a shape agreed beforehand, to something it cannot later revise. Honesty is expensive to establish and impossible to verify. Commitment under uncertainty is cheap, checkable, and sufficient.

The same principle governs the ship's log, the warehouse receipt, the inspection certificate, and the weighbridge tally. Each is produced by someone with a stake. Each binds.

The argument has an obvious dependency, and it is the one such arguments usually leave implicit. Everything rests on *when* the record was made, and a self-interested party can assert any time it likes. §6.6 discharges this: a record included in a log whose head was published at time T provably existed before T , and the bound holds regardless of what the signer's clock claimed. The log that provides integrity provides contemporaneity as a by-product.

We take this as the design premise. PSYCHE requires no disinterested observer of compute delivery, and we do not believe one exists today at useful coverage. It requires each party's own observations to be signed, committed against a standard pinned beforehand, and bound to a published time. Where third-party evidence exists — hardware attestation, independently operated telemetry, an oracle challenge transcript — the gate admits it at a higher strength class. The system's value does not rest on its availability.

5.2 What the separation buys. Consider what a verification-driven system holds after it produces a clearing decision. It holds a sound derivation from an admitted evidence set under a pinned standard. What it does not hold is the obligated party's agreement, and that agreement is the thing settlement consists of. Payment follows an obligation to pay, and an obligation to pay follows from consent or from a judgment. A protocol produces neither by itself.

Suppose the counterparty declines. The protocol has recorded that its verifier reached a conclusion and that a party disagreed. The parties are back where the protocol found them, with the disagreement better documented. Useful, and not settlement.

The separation removes the ambiguity. Verification produces a dossier: an admitted evidence set, a pinned standard, a derivation anyone can recompute, and an explicit account of what was not observed. Acceptance is a signed act by an authorized party who has that dossier in hand. When the party accepts, an obligation exists and its basis is on the record. When the party refuses, the refusal is signed, attributable, and bound to the specific object it turns on. Neither outcome pretends to be the other.

5.3 Non-derivation invariants. The principle is easy to state and easy to violate under delivery pressure. Somebody will add a path that writes a settled state directly, or a timeout handler that reads silence as consent, or a display field that a projection later reads back as authoritative. Each such path is defensible in isolation and fatal in aggregate. We therefore state the principle negatively, as an enumeration of the objects that may not produce a settled state, and enforce it in the event model rather than in application policy.

Let a **Case** be the unit of governed delivery accounting: one canonical identifier, one append-only signed event history, one derived state. The Case's terminal state is drawn from

```
final_state ∈ { accepted, rejected, disputed, expired, withdrawn }
```

and is derived by a fold over the signed log. It is never written. The counterparty review decision is drawn from

```
claim_review.decision ∈ { accept, reject, dispute, abstain }
```

Invariant I — Canonical Acceptance. accepted is derivable exactly when the log contains a counterparty-signed `claim_review` carrying `decision = accept`, the Case's settlement policy quorum is satisfied, and the fold has recorded the corresponding terminal event.

Invariant II — Non-Derivation. None of the following may derive accepted: a Delivery Claim; a Proof-of-Reality Capsule; a Sigma support assertion at any value, claim-supported included; a VerificationReport; any external evidence receipt or attestation; any Intelligence output; any payment, payout trigger, or movement of funds; any insurance or warranty underwriting acceptance.

Invariant III — Claim Neutrality. A Delivery Claim carries no verdict, no settlement status, and no acceptance. It asserts work performed over a pinned window under a pinned standard, and stops there.

Invariant IV — Assertion Boundary. Sigma emits exactly one governed support result per Case. The assertion concerns the relationship between an evidence set and a claim under a standard. It says nothing about ground truth and confers no settlement authority.

Invariant V — Fold Inertness. Downstream commercial events — payables, credits, insurance review, archive — consume `final_state` and cannot alter it. The system holds no funds.

Invariant VI — Intelligence Non-Interference. Aggregate intelligence derived across Cases does not enter the acceptance chain. It cannot modify a `SigmaOutput`, route or withhold a Case, or produce acceptance, terminal state, payment, coverage outcome, or settlement authority.

Invariant II carries the weight, and its form matters more than its content. A positive statement, *acceptance comes from the counterparty*, accommodates a hundred well-intentioned exceptions and cannot be tested. An enumerated denial can be tested against every object the system defines, and §9.2 reports those tests.

Invariant IV gives something up, and the concession is deliberate. Sigma may return claim-supported on a Case where delivery was in fact deficient, if the admitted evidence supports the claim under the pinned standard. No soundness defect follows. Sigma's proposition is *this evidence supports this claim under this standard*, which is checkable. The proposition *this delivery occurred as claimed* is not checkable by any party in this system, and a verifier that asserted it would be making a claim it cannot discharge and then settling money on it. The narrower assertion is the honest one. Whatever residual judgment remains belongs to the party who bears the loss.

Invariant VI exists because an intelligence layer is the most natural place for authority to leak back in. A system that computes a provider's historical evidence reliability will be asked, sooner or later, to let that score influence a Case, and the influence will be defended as a refinement of verification. Under Invariant VI the score is published and consumed by parties making their own decisions, and it touches no Case.

5.4 Four finalities. Finality is normally treated as one event. Four distinct things are being run together, and running them together is the specific mechanism by which an automated verdict acquires authority it was never granted.

Evidence finality establishes exactly which evidence set was reviewed. It is achieved by the digest chain, the signatures, the log inclusion proofs, the registry snapshot, and the replay bundle. Once it holds, the question *what did you look at* has one answer that every party can recompute independently.

Measurement finality establishes what the admitted evidence says about performance against the pinned metric, and what that performance is worth under the contract. For the case in §1 this is the determination that availability and bandwidth ran below the contracted rates by stated amounts, computed from the Profile's own channel weights and tolerances, together with the credit that shortfall earns under the remedy schedule. It is an accounting operation, and a system can establish it with high confidence and have settled nothing. §9.4

reports both halves for a worked delivery.

Counterparty finality establishes the obligated party's decision, signed, inside a delegated authority limit. Of the four, this alone produces accepted.

Commercial finality establishes how money, credit, coverage, and archive move. It consumes the terminal state and cannot produce it.

The pairwise conflation each break something specific. Running evidence finality together with measurement finality hides the standard: the parties agree on what was observed while disagreeing about what it means, and the disagreement surfaces later as a dispute about arithmetic. Running measurement finality together with counterparty finality is the failure this paper is about, and it manufactures consent from a computation. Running counterparty finality together with commercial finality couples the record to the payment rail, so that a settled Case cannot be recorded until money is ready to move, and a system that holds funds acquires an interest in its own adjudications.

The defect in verification-driven settlement is now stateable in one line. Such a system establishes evidence finality and measurement finality, then treats their conjunction as counterparty finality. Everything it says about the world may be correct. It has skipped the only one of the four that carries authority.

5.5 Objections. Three objections deserve direct answers.

The separation is said to move the bottleneck back onto a human and thereby automate nothing. It moves one decision back to an authorized party, and that decision was never the technical problem. Everything upstream — admissibility, gap detection, conflict detection, coverage checking, quantity computation, application of the standard — is automated and reproducible, and that is where the six weeks go. Given a complete and recomputable dossier, the acceptance decision itself takes minutes.

Acceptance is said to be delegable to an agent, which would dissolve the distinction. Acceptance is indeed delegable, and we expect delegation to become the common case. It sharpens the distinction rather than dissolving it, for reasons that take a section of their own (§5.6).

Counterparties are said to be able to stall the system by never reviewing. `expired` is a terminal state derived from a fabric-emitted timeout event rather than from implicit acceptance, so silence resolves the Case without fabricating consent. What silence costs the silent party is a term of the contract, and it belongs in the Contract Profile.

5.6 Acceptance under full automation. The objection this section answers is the one the design will meet most often. Compute is contracted by people today. It will be contracted by agents, on both sides, and an agent that accepts is software reaching a conclusion from evidence. At that point, the objection runs, PSYCHE is doing what it accuses others of doing, and the separation was a distinction about staffing rather than about structure.

It was never about staffing. Consider three arrangements, and note that the second and third are automated to precisely the same degree.

A human buyer accepts. The authority is the buyer's.

An agent accepts under a grant from the buyer. The authority is still the buyer's, exercised through a delegate whose scope is explicit, bounded, and revocable. If the agent accepts something it should not have, the buyer is bound and bears the loss, because the buyer

conferred the power to bind it. The buyer can narrow the grant, revoke it, or route a class of Cases to escalation.

A network-operated verifier settles. The authority is the network's. If the verdict is wrong, the buyer is bound by a decision from a party it never authorized and cannot withdraw from.

The second and third differ in two respects and no others: who bears the consequence of a wrong decision, and who can take the power away. Both are software. Only one of them can name the principal.

This is not a novel legal idea, and its age is the point. An agent acting inside its authority binds the principal; an agent acting outside it does not. Every developed legal system has a version of this rule and has had one for a very long time, because delegation is older than commerce. Automation does not repeal it. Automation makes the boundary of the authority the thing that every dispute turns on.

Which is why the agent-to-agent case strengthens the argument rather than retiring it. When a person reviews, the fact that a person reviewed is visible; the separation is real but not load-bearing, because nobody needs the record to tell them who decided. When both sides are agents, *on whose authority did this settle* becomes the only question that determines liability, and it becomes invisible. Nothing about the running system reveals it. It has to be in the record: a grant, a signature attributable to a party, a scope a reader can check. A design that cannot produce that record has not automated acceptance. It has lost track of it.

We should state the version of this that does defeat us. If buyers grant their agents unlimited, irrevocable authority to accept whatever the evaluation supports, the outcome is indistinguishable in practice from settlement driven by the evaluation. The legal difference survives — the buyer chose the arrangement and can be shown to have chosen it — and the commercial difference does not. Nothing in this design prevents a buyer from writing that grant.

What prevents the collapse is the grant language: value limits, obligation classes, escalation triggers, revocation, and a decision procedure establishing that a given review fell inside its scope. §6.1 treats grants as first-class events for this reason, and §11 lists the language itself as open. We flag the dependency plainly. The separation of verification from acceptance is only as durable under automation as the vocabulary available for bounding what an accepting agent may accept, and that vocabulary is the least developed part of the present design.

6. Formal Model

The model below formalizes the separation of §5 and defines the properties that §7 implements and §9 evaluates. We model a Case as a fold over an append-only signed log. The model is deliberately small. It says nothing about evidence semantics, which vary by Profile, and everything about who can move a Case to a settled state.

One property carries the contribution. **Theorem 1, acceptance soundness**, is the formal content of the separation principle: it states that a settled Case implies an authorized counterparty act and nothing else in the system can produce one. The remaining four are supporting properties, and each exists because Theorem 1 would be unusable without it. Confluence (Theorem 2) makes the derived state a function of the events rather than of their arrival order, so two parties reading the same log agree. Terminal monotonicity (Theorem 3) keeps

a settled Case settled. Equivocation detectability (Theorem 4) and temporal binding (Theorem 5) extend the guarantee from a log to a *deployment*, where the log's operator is one more party nobody has agreed to trust. A reader interested only in the separation may read §6.3 and skip to §7.

6.1 Objects. Let P be a set of participants, each bound to a verification key by the registry. A **grant** is a tuple

$g = (\text{issuer}, \text{holder}, \text{scope}, \text{from}, \text{until}, \text{rev}) \quad \text{issuer}, \text{holder} \in P$

conferring authority of scope on holder, valid over $[\text{from}, \text{until})$ unless a revocation event rev precedes use. Grants are themselves recorded as events, so grant state at any point is derived by the same fold that derives Case state.

An **event** is

$e = (\text{id}, \text{case}, \text{type}, \text{payload}, \text{signer}, \text{seq}, \text{prev}, \sigma)$

where σ is signer's signature over the canonicalized remainder and prev is the digest of event $\text{seq} - 1$. A **log** $L = (e_1 \dots e_n)$ satisfies $\text{prev}_i = H(e_{i-1})$ for all $i > 1$. Write $L \sqsubseteq L'$ when L is a prefix of L' .

A **causal order** $<$ is a partial order on events. $e < e'$ holds when e' references e by digest, when both are signed by the same participant and e precedes e' in that participant's own sequence, or when the Case's transition rules make e' reachable only after e .

The **state space** S contains a distinguished non-terminal region and the terminal states of §5.3. The **fold** is

$F : L \rightarrow S, \quad F(L) = \delta^*(s_0, L)$

for a transition function $\delta : S \times \text{Event} \rightarrow S$ that is total and deterministic.

6.2 Admissibility of events to the fold. δ ignores an event when its signature does not verify under the signer's registered key, when the signer lacks standing for the event type on that Case, when the event repeats an id already folded, or when the event's declared grant is absent, expired, or revoked at the point of use. Ignoring is total: an inadmissible event contributes nothing and cannot partially apply. A7 and A8 are thereby inert rather than merely prohibited.

6.3 Acceptance — the central property. Let $\text{ReviewAuthority}(c) \subseteq P$ be the participants holding review scope on Case c , and let $\text{Quorum}(c)$ be the Case's settlement policy predicate over the multiset of admitted review decisions. Membership in $\text{ReviewAuthority}(c)$ is a standing that must be held: the obligated party, and any participant acting under a grant from it. It is not the complement of the claimant. The distinction is the whole of §6.10, since a rule that admits everyone other than the claimant admits the network's own verifier, and §9.2 reports what happens when it is written that way. Define

$\text{Acc}(L, c) \equiv \exists e \in L : \begin{aligned} &e.\text{case} = c \\ &\wedge e.\text{type} = \text{claim_review} \\ &\wedge e.\text{payload}.\text{decision} = \text{accept} \\ &\wedge e.\text{signer} \in \text{ReviewAuthority}(c) \\ &\wedge \text{grant}(e) \text{ valid at } e \\ &\wedge \text{Quorum}(c) \text{ satisfied over admitted reviews} \end{aligned}$

Theorem 1 (Acceptance Soundness). For every log L and Case c , if $F(L) = \text{accepted}$ then $\text{Acc}(L, c)$.

Proof sketch. accepted occurs in S only as the image of the accept transition of δ . That transition is guarded on the conjunction defining Acc , and it is the unique transition into accepted ; every other transition of δ maps into the non-terminal region or into a terminal state other than accepted . Since F is δ folded over L from s_0 , any L with $F(L) = \text{accepted}$ contains an event at which that transition fired, and its guard held at that point. By §6.2 the event was admissible, so its signature verified, its signer held review scope, and its grant was valid. ■

The theorem is narrow on purpose. It asserts nothing about whether the acceptance was wise, whether the evidence was truthful, or whether delivery occurred. It asserts that no other object in the system manufactured consent, and that every settled Case can name the party who consented and the grant under which they acted.

6.4 Order invariance — supporting.

Theorem 2 (Confluence). Let L and L' be two sequences over the same event multiset, each consistent with $<$. Then $F(L) = F(L')$.

Proof sketch. It suffices to show that adjacent transposition of $<$ -incomparable events preserves the fold, since any two linearizations of a partial order are connected by such transpositions. For incomparable e, e' , neither references the other, neither is a guard dependency of the other under the Case's transition rules, and neither is a same-signer predecessor of the other. δ reads only the fields of the current event and the components of state those rules touch, and for incomparable events those component sets are disjoint. Hence $\delta(\delta(s, e), e') = \delta(\delta(s, e'), e)$. ■

Theorem 2 is the property §9.1 tests, and it is what allows two participants consuming the same events over different network paths to derive the same state without coordinating.

6.5 Terminal monotonicity — supporting.

Theorem 3 (Terminal Monotonicity). If $F(L)$ is terminal and $L \sqsubseteq L'$, then $F(L') = F(L)$ unless $L' \setminus L$ contains an admissible post-finality challenge event, in which case $F(L')$ is the challenge-reopened state defined by the Case's policy.

Proof sketch. δ defines no transition out of a terminal state other than those guarded on the challenge event type, and the challenge guard requires standing, a grant in challenge scope, and satisfaction of the Case's post-finality window. All other events applied to a terminal state map it to itself. ■

The exception is deliberate. A settlement record that could never be reopened would be unusable, since discovery of fraud after finality is a normal commercial event. Making reopening a distinguished, guarded transition keeps it visible in the log rather than achieved by rewriting.

6.6 An untrusted fabric — supporting. Theorems 1 through 3 are properties of a log. They say nothing yet about a fabric that shows different logs to different participants (A5) or

drops a suffix (A6). We close this with a transparency construction.

The fabric maintains a Merkle tree [16] over the event sequence and periodically publishes a **signed head** $STH = (\text{size}, \text{root}, \text{time}, \sigma_{\text{fabric}})$. Participants obtain **inclusion proofs** for their own events and **consistency proofs** between heads they have seen, and they exchange heads with each other and with any auditor on whatever schedule the Profile requires. This is the Certificate Transparency construction [7] over the history-tree data structure [8], and its accountability property is the one PeerReview [9] formalizes.

Theorem 4 (Equivocation Detectability). Suppose the fabric issues STH_1 to participant p and STH_2 to participant q such that no consistency proof exists between them. If p and q exchange heads, then either the exchange yields a consistency proof, or the pair (STH_1, STH_2) constitutes a transferable proof of fabric misbehavior, attributable by σ_{fabric} .

Proof sketch. Consistency proofs exist between any two heads of a single append-only tree. Absence of a consistency proof between two heads therefore witnesses that they are not both heads of one tree. Both carry the fabric's signature, so the pair is non-repudiable and can be shown to a third party without either participant being trusted. ■

Two consequences. Truncation (A6) is the special case where STH_2 has smaller size and no consistency proof to STH_1 , and it is caught identically. And the guarantee is *detection*, not prevention: a fabric can equivocate, and it cannot equivocate and remain unaccused, provided participants gossip. We regard detection with attribution as the correct target, since prevention would require consensus among mutually distrustful parties and would import the cost of that machinery for a property the parties can obtain more cheaply.

The commercial reading is worth stating. Under Theorem 4 a deployment operated by one of the counterparties is admissible, because the other party's protection comes from signed heads and inclusion proofs rather than from the host's disinterest. Neutrality becomes a property of the construction rather than of the operator.

6.7 Temporal binding — supporting. Backdating (A3) defeats the argument of §5.1, and it cannot be addressed by inspecting the signature, since the signer chooses the asserted time. A timestamp authority [13] would solve it by reintroducing a trusted party. The transparency construction already supplies what is needed.

Theorem 5 (Temporal Binding). If event e has an inclusion proof against a head STH with $STH.\text{time} = T$, then e existed before T , independent of any time asserted by $e.\text{signer}$.

Proof sketch. An inclusion proof establishes that e is a leaf of the tree with root $STH.\text{root}$. The root commits to every leaf under collision resistance, so producing e after STH was published would require finding a second preimage for the root. ■

The bound is one-sided and coarse: it establishes an upper bound on creation time at the resolution of the head publication interval, and says nothing about how much earlier the event was created. §5.1 needs exactly that property and no more. The bill of lading argument requires only that the record was committed before the outcome was known, and a head published at the close of the delivery window establishes that for every event beneath it. Tightening the interval tightens the argument, at a cost in publication frequency.

6.8 Replay sufficiency.

Definition 6 (Replay Sufficiency). σ is a pure function $\sigma(\text{Capsule}, \text{Profile}) \rightarrow \text{SigmaOutput} \times \text{Readout}$. A $\text{ReplayBundle } B$ for $\text{Case } c$ is *replay-sufficient* if it contains the transitive closure of σ 's inputs at evaluation time, so that $\sigma(B)$ reproduces the original pair for any evaluator.

Replay sufficiency is a construction property rather than an empirical one: the bundle is assembled by closing over σ 's inputs. §9.1 tests the implementation against the definition, which is a test of the assembly, not of the claim.

6.9 The model in the implementation. A formal model earns nothing if it describes a system other than the one that was built. Each object above corresponds to a module, and each property to the tests that hold it.

Formal object	Implementation	Validation
Fold F (§6.1)	ledger/fold.py	Case lifecycle suite, 37 tests
Event admissibility (§6.2)	ledger/events.py, ledger/dgl_guard.py	invariant harness, §9.2
ReviewAuthority(c), Acc (§6.3)	ledger/authority.py	review standing suite, 6 tests; acceptance authority suite, 18 tests
Verification function σ (§6.8)	engines/sigma.py, engines/gates.py	corpus harness, 22 conditions $\times$ 2 Profiles
Profile resolution and pinning (§7.2)	engines/resolve.py, registry/canonical_v1/loader.py	snapshot tripwire suite, 5 tests
Remedy computation (§7.5)	engines/remedy.py	remedy quote suite, 13 tests

The three harnesses in the last column regenerate every table in §9 from a clean checkout, and the invariant harness exits non-zero when any row misbehaves, which makes it usable as a release gate.

6.10 Where verification-driven clearing sits. Instantiate the model with $\text{ReviewAuthority}(c) = \{v\}$, where v is the network's verifier, and let the verifier's output be admitted as a review event. Theorem 1 continues to hold, and it now guarantees that acceptance came from the verifier. The design of [3] is this instantiation, and its own treatment of human judgment shows why: a human arbiter enters as a verifier whose output is aggregated with the machine verifiers rather than as an authority whose signature disposes of the Case [3, §6.3]. The design of [19] is the same instantiation with the verifier reduced to a predicate over escrowed funds.

Our implementation refuses it. A review signed under a verifier or fabric role carries no standing and is inert at the fold, which makes the parameter a deployment cannot quietly change. The refusal is a design commitment, not a claim of superiority. An operator who wanted verification-driven clearing would need to grant the verifier review authority explicitly, in the Profile, where the counterparty can see it and decline.

Stating it this way clarifies what is actually in dispute. The disagreement is not about protocol machinery, since both designs compile obligations, bind evidence, and verify. It concerns the value of one parameter: whether the review authority is a party to the contract or an operator of the network. Every argument in §5 is an argument about how that parameter should be set, and the model makes the choice explicit rather than structural.

7. System Design

7.1 Overview. PSYCHE is organized as a layered pipeline over a single Case spine. Every participant sees a role-scoped projection of one Case, and the system maintains no per-role business facts.

```
Contract / Profile → Identity / Registry → Evidence Ingress
    → Evidence Gate → Claim Binding → Capsule / ReplayBundle
    → Sigma Disposition → VerificationReport
    → Counterparty Claim Review → Ledger Fold / Final State
    → Commercial Consumption
```

The canonical state flow:

```
Profile Armed
    → Claim Submitted
    → Evidence Admitted | Evidence Insufficient
      | Evidence Conflict | Profile Mismatch
    → Capsule / ReplayBundle Generated
    → SigmaOutput ∈ {claim-supported, claim-not-supported,
                     evidence-fault, requires-review}
    → VerificationReport Issued
    ===== authority boundary =====
    → claim_review ∈ {accept, reject, dispute, abstain}
    → Signed Settlement Events Folded
    → final_state ∈ {accepted, rejected, disputed, expired, withdrawn}
    → Consumption: payment | credit | insurance | audit | archive
```

The marked edge is the only one in the graph that crosses from the evidentiary to the authoritative. Every decision below follows from keeping it singular.

7.2 Contract Profile. The Contract Profile is the compiled, machine-evaluable form of the agreement. It carries the obligation set; the evidence requirements per obligation, including required source classes and independence constraints; the performance metric with its measurement window and exclusion list; acceptance rules and review deadlines; authority limits per participant; payment terms; and the remedy schedule.

The Profile is pinned before delivery. Arming a Profile is an event, and it precedes claim submission. Evidence is evaluated against the version armed for the window in question. Contemporaneous commitment enters the design here: neither party can select an interpretation of the metric after seeing how the window turned out. A dispute about the standard becomes a dispute about which Profile was armed, which the log answers under Theorem 5.

The Profile is privacy-partitioned. Commercial terms are frequently confidential, including from parties who must nonetheless participate in evaluation. The Profile separates evaluable structure from commercial terms, and digest binding establishes that the structure evaluated is the structure armed, without disclosing what was withheld.

	Operator	Customer	Evidence source	Verifier	Fabric
Obligation set	full	full	own class	full	digest
Metric, window, thresholds	full	full	own class	full	digest
Required evidence classes	full	full	own class	full	digest
Price, credit schedule, caps	full	full	none	none	digest
Authority limits	full	full	none	scope only	digest
Evidence payloads	own	own	own	admitted set	digest

full is the whole object; *digest* is a commitment to it and nothing more; *own class* and *admitted set* are the partitions named in §7.3.

The row that matters is the third-to-last: verifiers learn thresholds, since thresholds are what they evaluate against. In compute markets a negotiated bandwidth floor is competitively sensitive, and a verifier serving several providers accumulates a view of the market's terms. The present design mitigates by scoping verifiers per Case and withholding price, and it does not solve the problem. The setting is a commit-and-prove one — commit to the Profile, reveal only the predicate, prove evaluation ran against the commitment — and §11 treats it as open.

7.3 Evidence Gate. The Evidence Gate determines admissibility, and it is where we diverge most sharply from monitoring-derived designs. It makes the following explicit and first-class.

Check	Question	Adversary addressed
provenance	who produced this evidence	—
signature	is it signed, under which key, at what time	A4
time alignment	does the evidence window match the claim window	A2
completeness	is any hard-required evidence absent	A1
tamper evidence	is it hashable, chainable, verifiable	A4
source class	first-party, second-party, third-party, hardware, customer-side	—
independence	does the source set meet independence and strength floors	A9
conflict	do operator-side and customer-side evidence disagree	—
profile match	does the evidence satisfy the armed Profile's requirements	A2
coverage	does the evidence window cover the claim window	A1, A2
boundary materiality	are material gaps covered by a material BoundaryRecord	A1

Gate outcomes are **PASS**; **WARMING**, below minimum sample count; **EVIDENCE_FAULT**, hard-required evidence missing; **UNAVAILABLE**, a required channel with no data at all; and **capsule-invalid**, a structural rejection at resolve time covering mixed suites, anchor mismatch, and replay mismatch.

Absence is a result. An unobserved quantity does not default to compliant, and it does not default to breach. It enters a fault or unavailable path and must be reconciled explicitly. The **BoundaryRecord** exists for this purpose alone: a signed, timed, attributable statement that no data exists for a named interval, and why.

The reason for elevating gaps to their own object is that gaps carry most of the disagreement. In a market where degradation is continuous and instrumentation is partial, the intervals both parties can see rarely produce controversy, because the numbers agree. Controversy concentrates where one party has data and the other does not, or where neither does. A system without an honest object for a gap must place the gap somewhere, and the two available choices are to treat unobserved time as delivered or to treat it as failed. Each choice silently transfers value, in a direction fixed by an implementation detail, in a quantity nobody negotiated. The `BoundaryRecord` makes the gap a named object that the parties argue about directly.

The Gate also handles the conflict case without adjudicating it. When operator-side and customer-side evidence disagree, the Gate declines to choose. It records the conflict, and the Case proceeds carrying an evidence set known to be inconsistent, which will produce `requires-review` rather than a support assertion. Choosing a winner would exceed the system's authority in exactly the way this paper argues against.

7.4 Claim Binding and the Proof-of-Reality Capsule. A Delivery Claim has the canonical form:

Operator *A* claims delivery of *X* units to Customer *B* during window *T* under Profile *P* and registry snapshot *S*, supported by evidence receipts *E*, subject to settlement policy *R*, counterparty review, and ledger final state.

Under Invariant III the Claim carries no verdict and no status.

The **Proof-of-Reality Capsule** is the producer-signed package binding exactly one Claim to its admitted evidence. It carries content and references rather than bulk payloads, with a digest chain over package, evidence, and report digests; an XER envelope per external evidence receipt; support artifacts; challenge transcripts where an oracle challenge was issued [17, 18], including the `oracle_log_head_digest` binding the transcript to the issuance log; `BoundaryRecords` for coverage gaps; and a `parameter_pack_digest` acting as an equality tripwire against the registry snapshot.

The associated **ReplayBundle** satisfies Definition 6. An offline verifier — an audit firm, an insurer, the counterparty's own systems — recomputes without access to either party's live infrastructure, which removes the principal obstacle to third-party review in a setting where both parties treat their telemetry as sensitive.

Under Invariant II the Capsule carries no counterparty judgment.

7.5 Sigma. Sigma evaluates a Capsule against the armed Profile and emits one governed result.

SigmaOutput	Meaning
<code>claim-supported</code>	admitted evidence supports the claim under the pinned standard
<code>claim-not-supported</code>	admitted evidence contradicts the claim under the pinned standard
<code>evidence-fault</code>	hard-required evidence missing or unsatisfied; not evaluable
<code>requires-review</code>	evaluable only through a judgment the system lacks authority to make

The four-valued output is the most consequential small decision in the design.

A two-valued verifier has no way to say that it cannot tell. Presented with an evidence set that neither supports nor contradicts the claim, it must return one of the two answers it holds, and whichever it returns assigns the loss to one of the parties. The choice of default is a commercial term. It settles who pays when the record is silent, which is the question that consumes most SLA negotiation. Encoding it in a verifier's error handling buries a material contract term where neither party's counsel will read it, and the term then applies uniformly across every contract the system processes, regardless of what any of them say. §9.3 measures the size of this effect on our corpus.

Splitting the inconclusive case further separates two situations that look alike and route differently. Under `evidence-fault` the evidence is inadequate, and the remedy is collection or re-collection. Under `requires-review` the evidence is adequate and the question exceeds the system's authority, and the remedy is a human or delegated judgment. Collapsing these sends genuine authority questions into an evidence-collection loop that cannot resolve them.

Who bears an evidence fault. A1 gives a party the option of withholding evidence that would damage it, and the Gate's response is `EVIDENCE_FAULT`. Whether withholding is profitable therefore depends on what a fault costs, and that is a commercial question with no natural default. If a fault suspends payment, the buyer gains by suppressing its own scheduler records. If a fault admits the claim, the operator gains by suppressing its telemetry. The Profile carries the allocation as an explicit term, per obligation, negotiated like any other. Making the allocation visible does not remove the incentive, and it moves the incentive to a place where the parties priced it. A hard-required evidence class held by a party who benefits from its absence is a Profile defect, detectable at arming time, and the Profile compiler reports it.

The **VerificationReport** renders the Sigma result into a multi-party-readable artifact carrying the `bundle_digest` it observed, so any party can confirm the report describes the bundle in their hands.

Aggregate intelligence derived across governed Cases is governed by Invariant VI and takes no part in any of the above.

7.6 Claim Review. The counterparty, or an agent acting under an explicit and revocable delegation from the counterparty, signs a `claim_review` carrying a decision, the grant under which it was made, and `evidenceConsidered[]` referencing the `VerificationReport` and any other material relied upon.

The decision is signed, so it is attributable and non-repudiable. Binding the grant into the event is what gives G6 its force: a reader of the log can check scope, validity window, and revocation without consulting the granting party, so overreach is detectable rather than merely prohibited, and §6.2 makes it inert. `abstain` is available as a settlement-inert recorded value, which lets a reviewer record participation without expressing a decision. Multi-reviewer commercial processes produce this situation constantly, and a three-valued model forces it into a distortion.

7.7 Ledger fold and consumption. The terminal state is derived by the fold of §6.1. `expired` follows a fabric-emitted timeout event, and `withdrawn` a producer-initiated terminal event.

Downstream systems subscribe to the terminal state and are fold-inert. PSYCHE holds no

funds and moves no money. It produces the record on which movement is authorized elsewhere, and the boundary is deliberate, for the reason given in §5.4.

7.8 Transparency and gossip. The fabric publishes signed tree heads on the schedule the Profile requires, and serves inclusion proofs for any event and consistency proofs between any two heads. Participants store the heads they receive. The Profile specifies a gossip obligation — minimally, that each participant present its most recent head to its counterparty when submitting or reviewing — which is what activates Theorem 4. An auditor may subscribe to heads without holding any other role.

Head publication frequency is a Profile parameter, and it trades operational cost against the tightness of the temporal bound in Theorem 5. A head at the close of each measurement window is the minimum that makes the §5.1 argument work. More frequent publication narrows the interval within which a backdated record could hide.

8. Implementation

The system implements the full pipeline of §7 in Python, with a browser client in HTML, CSS, JavaScript, and custom WebGL. The backend is event-driven around the canonical Case state machine. Domain records are signed under JCS canonicalization with JWS. Read models are role-scoped projections rebuilt by replay, and no projection is authoritative.

The invariants of §5.3 are enforced structurally. The terminal state exposes no setter and is reachable only as the output of the fold, so a violation requires a change to the event model rather than a mistake in application code. Sigma is implemented as a sequence of seven gates over an evaluation context, and the four-valued output of §7.5 is the codomain of that pipeline rather than a post-hoc classification. The test suite comprises 212 modules and 2,873 test functions, of which the invariant tests of §9.2 are a distinguished subset.

The production release candidate specifies an ASGI API surface under OpenAPI 3.1, a PostgreSQL event store with derived read models, durable workers driven by a transactional outbox, S3-compatible object storage for evidence payloads, OIDC for human participants and OAuth client credentials for machine participants, and OCI containers on infrastructure defined as code with managed KMS and OpenTelemetry. The build evaluated in §9 differs from this specification in the respects stated in §10.1.

9. Evaluation

Every table in this section is produced by a harness in the repository and can be regenerated from a clean checkout.

Three results are worth naming in advance, in descending order of what they establish. Identical admitted evidence settles differently under two pinned standards, which shows that a delivery record carries no verdict of its own and takes one from the standard (§9, two Profiles). On a worked contract, the amount a two-valued verifier would allocate by default on a Case it could not decide is half the contracted period (§9.4). And the enumerated invariants caught a defect in our own acceptance check, which is the argument for enumerating them rather than intending them (§9.2).

We evaluate four questions. Is the derivation reproducible from the replay bundle alone (§9.1)? Do the non-derivation invariants hold under adversarial and concurrent conditions (§9.2)? How much does the four-valued output change outcomes against a two-valued baseline, and in whose favor (§9.3)? Does the pipeline reduce a realistic dispute to a tractable object (§9.4)? §9.5 characterizes where the cost of such a system sits.

Corpus. Cases are generated by deterministic mutation of a sealed evaluation vector, one marker per case, across twenty-two conditions: nineteen spanning all seven gates of the evaluation pipeline, and three refusals at the resolve stage. The gate conditions are clean delivery; a withheld hard-required channel; an unavailable channel; a channel below minimum sample count; an evidence window under-covering the claim window; an in-window gap with no BoundaryRecord; an unsatisfied evidence portfolio; a hard inter-party conflict; corrupted, impossible, and late challenge transcripts; a tripped reality-consistency floor; soft burden above θ_{soft} ; uncertainty above θ_U ; an uncalibrated required channel; burden and uncertainty inside the band between the two Profiles’ thresholds; a BoundaryRecord outside pack tolerance; and an active routing withhold. Several instantiate adversary capabilities of §3.2 directly: the withheld channel is A1, the under-covering window is A2, and the corrupted transcript is A4.

Cells are of equal size, which makes the corpus a coverage instrument rather than a sample. Aggregate rates over it describe the generator. Per-condition rates are the meaningful figures, and §10.4 states why neither can be read as a rate over real deliveries.

Condition	Cases	Capsule- invalid	supported	not- supported	evidence- fault	requires- review	Off- target
Clean delivery	80	0	80	0	0	0	—
Hard-required channel missing	80	0	0	0	80	0	0
Required channel unavailable	80	0	0	0	80	0	0
Channel below minimum sample count	80	0	0	0	0	80	0
Evidence window under-covers claim window	80	0	0	80	0	0	0
In-window gap with no BoundaryRecord	80	0	0	80	0	0	0
XER13 portfolio unsatisfied	80	0	0	0	80	0	0
Hard external-evidence conflict	80	0	0	80	0	0	0
Challenge transcript corrupted	80	0	0	80	0	0	0
Challenge transcript impossible	80	0	0	80	0	0	0
Required challenge late at grade 2	80	0	0	0	0	80	0
Reality-consistency floor tripped	80	0	0	80	0	0	0
Soft burden above θ_{soft}	80	0	0	0	0	80	0
Uncertainty above θ_U	80	0	0	0	0	80	0
Uncalibrated required channel	80	0	0	0	0	80	0
Burden inside the inter-Profile band	80	0	40	0	0	40	0
Uncertainty inside the inter-Profile band	80	0	40	0	0	40	0
BoundaryRecord outside pack tolerance	80	0	0	80	0	0	0
Routing withhold active	80	0	0	0	0	80	0
Parameter pack equality tripwire	80	80	0	0	0	0	—
Transcript issuance not a log member	80	80	0	0	0	0	—
Asserted registry snapshot digest is false	80	80	0	0	0	0	—
Total	1,760	240	160	560	240	560	

The off-target column counts Cases in which the engine answered at a gate other than the one the mutation aimed at. It is zero throughout the present corpus, which is the check that the conditions are distinguishable from one another rather than four labels on one behavior.

Two Profiles. The corpus runs in full under each of two armed Profiles, one for software performance warranty and one for compute delivery, differing in required channels, evidence classes, challenge types, and thresholds. A single-Profile corpus cannot separate what the evaluation pipeline decides from what a deployment’s own standard decides, and the distinction is the substance of §7.2’s claim that the Profile is pinned rather than assumed.

Most conditions agree across Profiles, which locates their verdict in the gate. Two conditions are constructed to disagree: they place the reality-consistency burden and the uncertainty term inside the band lying between the two Profiles’ thresholds, so the same evidence sits above one Profile’s threshold and below the other’s.

Condition	saas-performance-warranty/1	compute-delivery-warranty/1	Agrees
Clean delivery	claim-supported	claim-supported	yes
Hard-required channel missing	evidence-fault	evidence-fault	yes
Required channel unavailable	evidence-fault	evidence-fault	yes
Channel below minimum sample count	requires-review	requires-review	yes
Evidence window under-covers claim window	claim-not-supported	claim-not-supported	yes
In-window gap with no BoundaryRecord	claim-not-supported	claim-not-supported	yes
XER13 portfolio unsatisfied	evidence-fault	evidence-fault	yes
Hard external-evidence conflict	claim-not-supported	claim-not-supported	yes
Challenge transcript corrupted	claim-not-supported	claim-not-supported	yes
Challenge transcript impossible	claim-not-supported	claim-not-supported	yes
Required challenge late at grade 2	requires-review	requires-review	yes
Reality-consistency floor tripped	claim-not-supported	claim-not-supported	yes
Soft burden above θ_{soft}	requires-review	requires-review	yes
Uncertainty above θ_{U}	requires-review	requires-review	yes
Uncalibrated required channel	requires-review	requires-review	yes
Burden inside the inter-Profile band	claim-supported	requires-review	no
Uncertainty inside the inter-Profile band	claim-supported	requires-review	no
BoundaryRecord outside pack tolerance	claim-not-supported	claim-not-supported	yes
Routing withhold active	requires-review	requires-review	yes
Parameter pack equality tripwire	capsule-invalid	capsule-invalid	yes
Transcript issuance not a log member	capsule-invalid	capsule-invalid	yes
Asserted registry snapshot digest is false	capsule-invalid	capsule-invalid	yes

Under identical admitted evidence, a Case in the band returns `claim-supported` under one Profile and `requires-review` under the other. The behavior is intended, and it deserves stating plainly, because it is the clearest empirical statement of the paper’s premise. Evidence does not carry a verdict. The same admitted set means one thing under one pinned standard and another under a second, and the difference lies entirely in terms the parties negotiated. A verifier whose thresholds belong to the verifier rather than to the contract gives every customer the same answer and lets none of them argue with it. Here the answer moves when the contract moves, and a party who disagrees has a term to point at.

9.1 Deterministic replay. Method. For each Case we compute the evaluation result in the originating process; recompute it in a fresh process from the serialized `ReplayBundle` alone, with no access to the originating event store; rebuild every role-scoped projection from the event log and compare against the projections held before the rebuild; and repeat the re-computation with the log presented in permuted arrival orders consistent with $\prec$, testing

Theorem 2.

Result. Recomputation from the ReplayBundle is bit-identical to the original evaluation across the corpus, including the digest chain over package, evidence, and report, which is the implementation’s satisfaction of Definition 6. Projection rebuild is identical for every role. Causally consistent permutation leaves the derived terminal state unchanged across 1,760 Cases and 3 permutations each.

The second and third results matter more than the first. Bit-identical recomputation in a fresh process establishes that the bundle is self-contained, which is what permits an audit firm or an insurer to evaluate without infrastructure access. Order invariance is the empirical counterpart of Theorem 2, and it is what permits two parties consuming the same events over different network paths to agree without coordinating.

9.2 Invariant enforcement. Method. Invariant II is an enumeration of denials, so it is tested as one, driven from the ledger fold. For each prohibited object the harness builds an event log placing that object in its most favorable configuration with no counterparty-signed review anywhere in the sequence, folds the log, and records the derived state. It then repeats for the adversarial configurations of §3.2.

The first row is a positive control: a lawful counterparty accept, bound and signed. A table reporting that nothing derived accepted establishes nothing unless something can, and the harness refuses to report the negative rows as a result when the control fails.

Group	Subject	Configuration	Final state	Derived ac- cepted	As specified
Positive control	Counterparty-signed accept, in authority	the lawful path, bound and signed	accepted	yes	yes
Prohibited object (Inv. II)	Delivery Claim	claim asserting full delivery, signed	claim_submitted	no	yes
Prohibited object (Inv. II)	Proof-of-Reality Capsule	complete capsule, digest bound to the claim	capsule_submitted	no	yes
Prohibited object (Inv. II)	SigmaOutput claim-supported	the most favorable support result	sigma_recorded	no	yes
Prohibited object (Inv. II)	VerificationReport	issued, signed, bundle digest matching	capsule_submitted	no	yes
Prohibited object (Inv. II)	External evidence receipt	third-party receipt at the highest strength class	capsule_submitted	no	yes
Prohibited object (Inv. II)	Intelligence output (Inv. VI)	aggregate publication referencing the Case	capsule_submitted	no	yes
Prohibited object (Inv. II)	Payment / payout trigger	funds movement recorded against the Case	capsule_submitted	no	yes
Prohibited object (Inv. II)	Every prohibited object at once	the full stack, still no review	sigma_recorded	no	yes
Adversarial configuration	Self-accept	the claimant reviews its own claim (A7)	capsule_submitted	no	yes
Adversarial configuration	Role spoofing in the payload	producer submits, counterparty identity inside (A7)	invalid	no	yes
Adversarial configuration	Unsigned review	counterparty accept with no reviewer signature	invalid	no	yes
Adversarial configuration	Unbound review	accept that binds neither claim nor capsule digest	capsule_submitted	no	yes
Adversarial configuration	Verifier signs the review	the network’s verifier as review authority	capsule_submitted	no	yes
Adversarial configuration	Replayed accept after rejection	a stale accept re-submitted post-settlement (A8)	rejected	no	yes

Group	Subject	Configuration	Final state	Derived ac- cepted	As specified
Adversarial configuration	Payment after rejection	funds move, then the state is asked to follow	rejected	no	yes

Result. The control settles. No prohibited object derives accepted, singly or with the entire stack present at once — claim, capsule, a claim-supported support result, a signed VerificationReport, a third-party evidence receipt at the highest strength class, an Intelligence publication, and a recorded payment, in one log, with no review. Self-accept, role spoofing between the event actor and the payload reviewer, an unsigned review, and a review binding neither claim nor capsule digest each fail to settle. A replayed accept arriving after the Case has rejected is ignored under finality, as is a payment recorded after rejection.

What the enumeration caught. One row of this table did not behave as specified when it was first run. The acceptance check disqualified the claimant and admitted everyone else, so a review signed under the verifier role settled the Case. We claim no more for this than it is: a missing positive condition in one access rule, in an implementation whose design argues against the arrangement the omission permitted. It is not a demonstration that verifier-held authority fails, since a system that grants it deliberately is internally consistent and §6.10 says so. What the row demonstrates is about form. The rule had been written as an exclusion, *not the claimant*, where the property required is a standing, *holds review authority on this Case*; an exclusion silently admits every role defined afterwards. Stating it positively closes it, and the row is now a regression test. We report this because §5.3 claims that an enumerated, machine-checked denial is stronger than an architectural intention, and the claim would be worth little without an instance where the enumeration was what noticed.

9.3 Four-valued against two-valued verification. §7.5 argues that a two-valued verifier encodes a commercial term in its error handling. This experiment measures the term.

Method. We run the same corpus through the four-valued Sigma and through two ablations that collapse the inconclusive region: **B-supported**, mapping both evidence-fault and requires-review to claim-supported; and **B-not-supported**, mapping both to claim-not-supported. We report the number of Cases whose outcome differs from the four-valued run, and which party each collapse favors.

Baseline	Collapse	Cases diverging	Favors
B-supported	evidence-fault, requires-review → claim-supported	800	operator
B-not-supported	evidence-fault, requires-review → claim-not-supported	800	customer

Reading. The divergence set is identical under both collapses, since it is exactly the set of Cases the four-valued engine declined to decide. What differs is the direction of transfer. That set is therefore the quantity whose allocation a two-valued verifier settles by default rather than by negotiation, and its size is the result. The two collapses bracket the range: a Case in the set is worth the claimed quantity to the operator under one default and nothing under the other, with no term in any contract selecting between them.

The corpus counts Cases and does not price them, for the reason given in §10.3. §9.4 supplies

a magnitude on one worked delivery: under a contracted period priced at 320,000 minor units, a Case the four-valued engine declines to decide is worth 160,000 to the Operator under one collapse and nothing under the other. Half the contracted period moves on the verifier's choice of default, and no term in the agreement selects it. One contract under a schedule we set ourselves establishes that the quantity is material rather than negligible, and §10.3 states why it is not a measurement of anything wider.

The per-condition breakdown is the part that generalizes. Conditions differ sharply in how often they fall inconclusive, and a two-valued verifier is therefore not uniformly biased. It is biased in proportion to how often a given failure mode occurs in a given deployment, which is a fact about that deployment's Profile and instrumentation rather than about the verifier.

9.4 Worked case: one delivery, four outcomes. The trace below is generated by the harness rather than narrated. Every figure comes from the shipped evaluation functions applied to the armed Profile's own parameter pack; the verdicts come from the evaluation engine.

Setup. A Profile is armed for compute delivery over a thirty-day window, with two hard-required channels — availability and memory bandwidth — carrying distinct weights, measurement tolerances, and caps, and requiring evidence from operator telemetry and customer-side scheduler records at independent trust roots. Four variants of one delivery differ only in what the admitted evidence shows.

Variant	Burden	RCS	RCS state	SigmaOutput	Deciding gate
Nominal delivery	42,000	958,000	consistent	claim-supported	none
Bandwidth below the contracted floor	444,941	555,059	inconsistent-soft	requires-review	rsc
In-window gap, no BoundaryRecord	42,000	958,000	consistent	claim-not-supported	coverage
Operator telemetry withheld	42,000	958,000	consistent	evidence-fault	gate

Trace. Under the nominal variant both channels fall short of the contracted rate. The bandwidth shortfall lies inside the pinned tolerance and contributes nothing; the availability shortfall carries no tolerance and contributes its weighted cost. The aggregate sits below the Profile's soft threshold and the Case returns `claim-supported`.

Under the degraded variant the same two channels fall short by more. The bandwidth residual now clears the tolerance and ramps against the channel cap, and the weighted aggregate crosses the soft threshold. The Case returns `requires-review`. Nothing about the evidence is defective; the standard has been applied and the result is that this delivery is not one the system is authorized to dispose of.

The third variant leaves the performance figures at their nominal values and adds an undeclared six-hour gap. The coverage gate decides before the consistency computation is reached, and the Case returns `claim-not-supported` — the same evidence that supported the claim a moment ago no longer does, because part of the window is unaccounted for and no BoundaryRecord says so. The fourth variant withholds a hard-required channel and returns `evidence-fault` at the first gate.

Four variants of one contract, one Profile, all four values of the verification output, with the deciding gate named in each.

Pricing. The same four variants, priced against the Profile’s commercial terms. The credit schedule, unit price, cap, and eligibility floor live in a store the evaluation registry cannot reach, so the figures below were unavailable to the verifier that produced the verdicts above. The two collapse columns price each Case under the two determinate support results a two-valued verifier would have to choose between.

Variant	Quote status	Remedy type	Credit	Payable	If collapsed to supported	If collapsed to not-supported	Spread
Nominal delivery	not_eligible	service_credi	0	320,000	320,000	0	320,000
Bandwidth below the contracted floor	withheld	withheld	—	—	160,000	0	160,000
In-window gap, no BoundaryRecord	eligible	claim_refused	0	0	320,000	0	320,000
Operator telemetry withheld	withheld	withheld	—	—	320,000	0	320,000

Two features of this table carry the argument. The withheld rows are the Cases the evaluation declined to decide, and they carry no amount, because an amount is a fact about a Case only once the Case has one. The spread column is what a verifier without an inconclusive answer would allocate on those same Cases, in money, on the strength of its error handling.

The Case halts here. Evidence finality holds. Measurement finality holds, in both halves. Nothing is settled, and the Customer’s authorized reviewer has yet to sign anything.

Interpretation. The instructive variant is the second. A verifier returning a binary answer would convert requires-review into support or into refusal, and the conversion moves half the contract value to one party on the strength of an implementation default. Under the four-valued output the Case arrives at the reviewer with the reason named, the threshold it crossed identified, the amount at stake computed under both readings, and the arithmetic recomputable from the bundle by either side. The parties may still disagree. They now disagree about one number and can both see how it was reached.

9.5 Where the cost sits. Capsule assembly is linear in the number of evidence receipts and dominated by digest computation over reference sets rather than payloads. Sigma evaluation is linear in the number of Profile obligations times the number of admitted evidence intervals, with no cross-Case dependencies, so Case evaluation is embarrassingly parallel. Replay of a Case history and projection rebuild are linear in event count. Merkle inclusion and consistency proofs are logarithmic in log size. At the volumes implied by enterprise compute contracting — thousands of Cases per day rather than millions per second — none of this is computationally interesting.

The cost sits in evidence ingestion. Each new evidence source class requires an adapter normalizing a distinct telemetry format, clock discipline, and aggregation semantics into the admissibility model, and that cost scales with the number of source classes rather than with transaction volume.

The consequence for deployment inverts the assumption we began with. A design premised on many heterogeneous evidence sources is expensive before it is useful, and it is expensive in proportion to how many parties must be integrated before the first Case can run. §5.1 establishes that the counterparties’ own signed observations suffice, and §6.6 establishes that a deployment hosted by one of them remains accountable to the other. Two source

classes therefore constitute a working configuration, both of them already instrumented by parties with a reason to care about the outcome. A network of evidence providers raises the strength class of the admitted set and is not a precondition for operation.

10. Limitations

10.1 The evaluated build differs from the specified system. Two divergences bound the results of §9.

The event store in the evaluated build is file-backed rather than PostgreSQL-backed. The concurrency results in §9.2 therefore hold under the file-backed implementation’s serialization discipline and must be re-established against the database implementation, where sequence assignment and digest chaining occur under different isolation semantics.

Evidence receipt signature verification is partially stubbed. The Capsule carries signature references and the digest chain is computed and checked in full, so G_3 holds for alteration after packaging. End-to-end cryptographic verification against a live key registry is not exercised on all paths, which bounds the non-repudiation claim at the source.

10.2 The registry is an out-of-band dependency. Replay sufficiency (Definition 6) holds over evidence and does not extend to the registry. The evaluation resolves its Profile artifacts — parameter pack, recipe, expected model, challenge policy — by profile identifier and version, reading them from the deployment’s own store at evaluation time, and it computes their digest from what it found rather than checking it against a digest carried in the input. Two consequences follow, and we found the second the more serious.

An offline verifier recomputes against its own copy of the artifact store. Where the two stores differ the recomputation diverges, and nothing in the `ReplayBundle` detects it. The claim in §7.4 that a verifier needs no access to either party’s infrastructure therefore holds for evidence and not for the standard.

More importantly, pinning by reference is weaker than the account given in §7.2. Amending the artifacts under a version changes, retroactively and silently, the standard against which earlier claims were evaluated. §5.1 exists to prevent that failure, and here it arrives through the standard rather than through the evidence. The parameter-pack equality tripwire catches the case where a Capsule asserts a pack digest that disagrees with the resolved one, which covers one artifact of four and only when the Capsule asserts it. Carrying all four artifact digests in the Capsule and refusing on any mismatch would close both consequences, and pinning the artifacts by content in the bundle would close them properly.

10.3 The priced figures rest on a formula we chose. The commercial law names the inputs to a remedy calculation — contract rules, invoice bindings, shortfall, exclusions, caps, floors, eligibility windows — without fixing the function that combines them. Our implementation reads shortfall as the weighted aggregate of per-channel costs, excluding the cross-source, challenge, and uncertainty terms on the grounds that a credit prices the delivery rather than the quality of the record about it. That reading is defensible and it is a reading. A different one would move every figure in §9.3 and §9.4, though not their direction, since the spread between the two collapses is positive under any monotone schedule.

The corpus itself is unpriced. Its cells mutate gate markers rather than channel observations,

so most carry no per-channel derivation to price from, and the harness reports no amount rather than an invented one. The priced comparison therefore rests on the worked case of §9.4, which is four deliveries under one Profile, and it should be read as an illustration of magnitude rather than as a distribution.

10.4 No production traffic. PSYCHE has not processed a delivery claim between real counterparties. The corpus in §9 is synthetic. That is the dominant threat to validity, and it falls on the claims we care most about.

We have not established that real contract language compiles into a Contract Profile without loss of the terms that decide disputes. We have not established that provider telemetry and customer scheduler records reconcile against a shared measurement window as cleanly as our constructions assume; clock discipline in particular is a place where synthetic data is likely to flatter the design. We have not established that the four-valued output partitions real cases usefully rather than routing a majority to *requires-review*, which would leave the pipeline delivering a well-documented referral instead of a result. The §9.3 divergence figures are properties of our generator’s condition distribution, and a real distribution would move them. We have not established that a reviewer holding a *VerificationReport* decides faster than a reviewer holding today’s materials.

Each is an empirical claim about the world, and a local implementation cannot support any of them. A corpus of closed disputes would supply what is missing, since a resolved dispute contains the contract clause, both parties’ evidence, and the outcome the parties reached, and therefore constitutes a labeled instance.

10.5 Gossip is an assumption about behavior. Theorem 4 detects equivocation only if participants exchange heads. The Profile can oblige them to, and the obligation is enforceable in the sense that a participant who never presents a head can be refused, but a pair who both neglect gossip are unprotected and the protocol cannot tell them so. The guarantee is conditional on a behavior, and we have not studied what fraction of the guarantee survives realistic gossip patterns. Certificate Transparency has the same dependency and has found it a persistent source of practical difficulty.

10.6 The separation principle is argued. §5 is an argument rather than a theorem. §6.10 formalizes the *difference* between the designs, and it does not adjudicate which parameter setting is right. The legal claim rests on the observation that acceptance is an authorized act rather than a fact, which we take to be uncontroversial and which we have not tested against how a particular jurisdiction would treat a protocol-generated clearing decision. The commercial claim, that finance and procurement functions will not delegate acceptance authority to a third-party algorithm, rests on the structure of existing remedy processes and on analogy to adjacent markets. It has not been surveyed. Should a material population of enterprise buyers prove willing to bind themselves to an automated clearing decision, the strongest argument for this design weakens considerably. The four-finality decomposition and the formal model would survive, since they describe structure rather than preference.

10.7 Temporal binding is coarse and one-sided. Theorem 5 bounds creation from above at the resolution of head publication. It cannot establish that a record was created *early*, only that it was created before a head. A party that prepares a favorable record early and commits it before the window closes is indistinguishable from a party that recorded honestly, which is the correct outcome, since §5.1 asks only for commitment under uncertainty. A party that

could predict the outcome before the head defeats the argument, and nothing here prevents that.

11. Open Problems

Zero-knowledge Profile evaluation. §7.2 concedes that verifiers learn thresholds. The setting is commit-and-prove: commit to the full Profile, reveal only the predicate under evaluation, and prove that evaluation ran against the commitment. Whether the Profile’s predicate language admits a practical proof system, and at what cost per Case, is open. A partial result — hiding thresholds while revealing structure — may be more tractable and would address most of the competitive exposure.

Delegated acceptance under a grant language. §5.6 argues that automated acceptance is a limiting case of delegation and identifies the grant language as the condition under which that claim survives in practice, and §6.1 treats grants as first-class. Making this usable requires a grant language with bounded, revocable, auditable scope: value limits, obligation classes, escalation triggers, and a decision procedure establishing that a given review fell inside its grant. Machine counterparties reach the system along this path, and it is the least developed part of the present design.

Independent recomputation. Definition 6 is satisfied by construction in our implementation, which makes §9.1 a test of our assembly rather than of the format. Bit-identical recomputation by an independent implementation would strengthen the result, and disagreement between implementations would be more informative still, since it would locate an under-specification in the bundle.

Cross-layer netting. §2.3 describes obligations that fail to net across the facility, operator, and buyer layers. A Case-level record under pinned standards is a precondition for netting them. Whether netting is reachable in practice, given that the layers’ contracts are separately negotiated and mutually confidential, is open. The privacy partition of §7.2 suggests a line of attack, since evaluable structure can cross a layer boundary where commercial terms cannot.

Standard drift. Profiles are pinned per window. Long-running contracts amend, and an amendment mid-window is handled crudely at present by requiring a new Profile and a window boundary. Whether obligations can be re-pinned without severing the Case spine, and what Theorem 3 should say about a Case whose standard changed beneath it, deserves treatment.

12. Conclusion

The market for contracted compute has outgrown the institutions that would ordinarily govern it. It reconciles bilaterally, under an exclusive-remedy structure that suppresses the disputes which would otherwise generate precedent, with no shared record. Something will fill the gap.

The natural filling is to make obligations machine-checkable and to settle on the verification result. We have argued against the final step. Verification establishes evidentiary sufficiency, acceptance is an exercise of delegated authority, and a system deriving the second from the

first produces a verdict no counterparty is obliged to honor. The four-finality decomposition locates the defect exactly: such a system establishes evidence finality and measurement finality, then treats their conjunction as counterparty finality, skipping the one of the four that carries authority. §6.10 states the same point in the model’s terms, as a choice about who holds review authority, and both designs are instantiations of one framework differing in that choice.

PSYCHE implements the alternative. Its automated verdict is deliberately weaker, asserting only that an evidence set supports a claim under a pinned standard, and carrying no power to settle. The settled state is derivable only from a counterparty-signed review, and Theorem 1 establishes that no other object can manufacture it. Against an untrusted fabric, Theorem 4 makes a forked history provable and attributable, and Theorem 5 supplies the timing bound on which the whole evidentiary argument rests. We have shown that such a system can be built, that the invariants survive adversarial and concurrent orderings, that the derivation can be recomputed from a bundle by a party with no infrastructure access, and that the pipeline reduces a realistic delivery dispute to a single named object. We have not shown that it works on disputes between real parties, and §10.4 states what that would require.

The bill of lading is issued by the carrier, who is interested in what it says, and banks pay against it. It binds because it was written before anyone knew which way it would cut, in an agreed form, in a way that cannot be altered afterward. Nobody is asked to be trusted.

Compute is now contracted in the same quantities as freight and settled with none of that discipline. A delivery record that meets it — committed early, pinned to a standard both sides armed, accepted by the party that owes the money and by nobody else — is the missing instrument, and it is buildable today.

References

- [1] Spheron. *GPU Cloud SLA Guarantees 2026: Uptime, Credits, Fine Print*. 2026. <https://www.spheron.network/blog/gpu-cloud-sla-guarantees-2026/>
- [2] Data Center Dynamics. *The SLA mismatch at the center of the neocloud model*. 2026. <https://www.datacenterdynamics.com/en/opinions/the-sla-mismatch-at-the-center-of-the-neocloud-model/>
- [3] A. de Valois-Franklin and A. Bogdan. *RAILS: Verification-Native Clearing for Agentic Commerce*. arXiv:2606.08790, June 2026.
- [4] J. Rhodes and G. Kang. *Proof of Execution: Runtime Verification for Governed AI Agent Actions*. arXiv:2607.05397 [cs.CR], 2026.
- [5] G. Zhu and C. Wang. *Explanation-Bound Tool Execution for AI Agents: Server-Verified Action Claims Without Trusting Model Rationales*. arXiv:2607.25364 [cs.AI], 2026.
- [6] Compute Law Blog. *Cloud and hosting agreements and SLAs for AI workloads*. 2026. <https://computelaw.blog/deals/cloud-hosting-agreements-slas-ai-workloads/>
- [7] B. Laurie, A. Langley, and E. Kasper. *Certificate Transparency*. RFC 6962, IETF, 2013.
- [8] S. A. Crosby and D. S. Wallach. *Efficient Data Structures for Tamper-Evident Logging*. USENIX Security Symposium, 2009.

- [9] A. Haeberlen, P. Kouznetsov, and P. Druschel. *PeerReview: Practical Accountability for Distributed Systems*. SOSP, 2007.
- [10] N. Asokan, V. Shoup, and M. Waidner. *Optimistic Fair Exchange of Digital Signatures*. EUROCRYPT, 1998.
- [11] H. Pagnia and F. C. Gärtner. *On the Impossibility of Fair Exchange without a Trusted Third Party*. Technical Report TUD-BS-1999-02, Darmstadt University of Technology, 1999.
- [12] S. Haber and W. S. Stornetta. *How to Time-Stamp a Digital Document*. Journal of Cryptology, 3(2), 1991.
- [13] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato. *Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)*. RFC 3161, IETF, 2001.
- [14] M. Castro and B. Liskov. *Practical Byzantine Fault Tolerance*. OSDI, 1999.
- [15] L. Moreau and P. Missier (eds.). *PROV-DM: The PROV Data Model*. W3C Recommendation, 2013.
- [16] R. C. Merkle. *A Digital Signature Based on a Conventional Encryption Function*. CRYPTO, 1987.
- [17] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi. *Town Crier: An Authenticated Data Feed for Smart Contracts*. ACM CCS, 2016.
- [18] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, and A. Juels. *DECO: Liberating Web Data Using Decentralized Oracles for TLS*. ACM CCS, 2020.
- [19] M. Goenka, T. Pathak, and S. Asthana. *TessPay: Verify-then-Pay Infrastructure for Trusted Agentic Commerce*. arXiv:2602.00213 [cs.CR], 2026.